

## 회고 — 배포 검증 위치

배포한 게 진짜 반영됐는지 자동으로 확인하는 도구를 만들었습니다. 배포 후 사이트를 새로고침하며 눈으로 확인하는 일이 반복이었고, 200 OK 만 보고 넘어갔다가 CDN 캐시 때문에 사용자만 옛 화면을 보던 사고를 겪었습니다. 그래서 살아있는지·내용이 맞는지·**사용자 화면까지 반영됐는지**를 5 개 층으로 확인하고 상태가 바뀔 때만 텔레그램으로 알리는 프로그램을 만들었습니다.

---

### 1. 어떤 도구를 어떤 순서로 썼나 (s2a)

- ① 주제 발굴                      Claude Fable 5 · 웹 검색  
내 실제 반복 작업 목록화 → "배포 후 확인"으로 좁힘
- ② 설계 문답                      Claude Fable 5  
내가 묻고 AI 가 답하는 게 아니라, AI 가 결정 지점을 물어오게 시킴  
→ 4 가지 결정을 잠그고 스펙 문서로 박음
- ③ 구현                              Claude Opus 5 / Fable 5  
한 번에 한 슬라이스 (감지 층 → 알림 → 상태 전이 → 배포 모드 → 보안 층)
- ④ 교차 검증 ★                    Codex gpt-5.6-sol (reasoning xhigh)  
Claude 가 짠 것을 다른 모델이 감사. 20 라운드.  
지적마다 내가 수용/기각 판단, 기각은 file:line 근거
- ⑤ 검사자 실험                    Codex gpt-5.6-sol  
같은 결함에 프롬프트 형태만 바꿔 검출률 측정
- ⑥ 기록                              매 세션 실시간 (사후 몰아쓰기 금지)

③과 ④ 사이를 20 번 왕복한 게 이 과제의 실체입니다. 만드는 데 쓴 시간보다 "이게 진짜 맞나"를 확인하는 데 쓴 시간이 훨씬 많습니다.

가장 효과가 컸던 건 ④ **작성자**와 **검사자**를 다른 모델로 분리한 것입니다. 같은 모델에게 자기 코드를 검토시키면 자기 결론을 방어합니다 — 실제로 관측했습니다. 검사 세션을 이어서 호출했더니 두 번째 질문에 \*계측 결과도 이 순서였고 테스트가 통과했습니다. 현재

구현 변경은 필요하지 않습니다"\*라며 앞선 자기 답을 근거로 결함을 부정했습니다. 그래서 매 호출 세션을 리셋했습니다.

---

## 2. 막힌 지점과 푼 방법 (s2b)

### 2-1. 같은 오탐이 다섯 번 반복됐다

정상 사이트(배너가 요청마다 바뀌는 페이지)를 "해킹당했다"고 오판하는 문제가 13~17 차에 걸쳐 네 번 수리하고 네 번 다시 뚫렸습니다. 매번 다른 조건에서요.

**푼 방법:** 다섯 라운드를 나란히 놓고 공통점을 찾았습니다. 전부 **"한 번에 표본을 어떻게 배치할까"** 한 축 위에 있었습니다. 정해진 패턴으로 유한하게 관측하면 그 길이와 맞물리는 주기에서 **반드시** 어긋납니다 — 이 축 위에는 답이 없었습니다.

수리를 정교화하는 대신 **판정 축을 3 개로 쪼갰습니다**: ① 페이지가 회전하는지 먼저 관측 ② 한 번의 관측으로 확정하지 않고 여러 패스에 걸쳐 누적 ③ 매번 표본 수를 다르게 뽑아 위상 고정을 없앴.

결과: 주기  $2 \sim 24 \times$  시작 위상 전수 1,206 패스에서 **오탐 86 건  $\rightarrow$  0 건**, 진짜 클로킹 탐지는 유지.

### 2-2. 수리가 다른 걸 망가뜨렸다 (두 번 연속)

축을 바꾼 직후 게이트가 **제가 만든 새 결함**을 찾았습니다. 오탐을 막으려다 **앞 라운드에서 닫아둔 미탐을 다시 연** 것이었고, 그 장치의 존재가 **제가 지운 함수의 주석에 적혀** 있었습니다.

그걸 고쳤더니 다음 라운드가 또 잡았습니다. 이번엔 벽을 없앤 게 아니라 **옳기기만** 했습니다 — 무작위화를 두 경로 중 한쪽에만 걸었기 때문인데, 그 법칙은 **제가 같은 파일 첫머리에 적어둔 문장**이었습니다.

**푼 방법:** 매 수리마다 "이 수리가 무엇을 깨뜨릴 수 있나"를 파라미터 구간 전체에서 확인. 그리고 없앨 수 없는 한계는 숨기지 않고 **계산 가능하게** 만들어 README 에 실측 표로 넣었습니다.

### 2-3. 도구 없이는 못 하는 일을 만났다

실행 화면 녹화를 제가 직접 하려 했는데, 에이전트 셀에서 띄운 창이 화면에 안 붙었습니다. "제가 못 합니다"라고 보고했더니 **"화면캡처 도구 받아와서 해라"**는 지시를 받았고, 받아서 해보니 **창 영역 캡처는 됐습니다.**

그런데 리허설 영상을 프레임으로 뽑아 **눈으로 확인**했더니, 화면에 다른 작업 창과 메신저가 그대로 찍혀 있었습니다. 그대로 제출했으면 공개 제출물에 무관한 정보가 들어갔습니다.

**폰 방법:** 즉시 폐기하고, 녹화기가 **자기 창이 최상위일 때만** 촬영하고 아니면 중단하도록 가드를 넣었습니다(바탕화면 전체로 대체하는 경로는 아예 제거). 순서도 맨 뒤로 옮겼습니다 — 깨끗한 화면이 전제이므로.

*"못 한다"는 보고는 **도구가 없다는 뜻이지 불가능하다는 뜻이 아니었습니다.** 그걸 밀어붙인 게 능력 확인과 사고 예방을 동시에 만들었습니다.*

---

## 3. AI 답이 틀리거나 마음에 안 들 때 어떻게 했나 (s2c)

### 규칙 1 — "했습니다" 앞에는 반드시 실측을 붙인다

가장 자주 쓴 규칙이고 가장 많이 잡았습니다. 완료 보고 전에 테스트·라이브 확인·해시 대조를 실제로 돌리고 **출력을 그대로** 붙였습니다. 이 규칙이 없었으면 "수리 완료" 보고 중 몇 건은 거짓인 채로 넘어갔을 겁니다(실제로 한 번 적발돼 정정했습니다).

### 규칙 2 — 검사자를 맹목 수용하지 않는다

Codex 지적을 그대로 받지 않고 **하나씩 재현해서** 수용/기각을 판단했습니다. 기각할 땐 file:line 근거를 댔습니다. 실제로 기각한 예:

- "시간축으로 회전하면 오탐이 난다" → 요청 순서가 구조적으로 막는 것을 6만 패스로 확인 후 기각
- 등급 과대 주장(P1 주장이 실제 P3)은 강등

반대로 검사자가 맞고 제가 틀린 경우가 훨씬 많았고, 그건 그대로 기록했습니다.

### 규칙 3 — 가드를 실제로 떼서 RED 를 본다 (음성 대조)

"이 방어 장치가 작동한다"는 주장은 **장치를 제거하면 실패하는지**로만 증명됩니다. 이 규칙이 두 건을 잡았습니다:

- 회귀 테스트가 결함 코드에서도 통과하는 **실패 불가** 구조였던 것
- **제가 만든 음성 대조 자체가** 아무것도 제거하지 않고 통과하던 것

두 번째가 특히 뼈아팠습니다. 검사 장치도 검사해야 한다는 걸 그때 배웠습니다.

### 규칙 4 — 방향은 AI 에게 맡기지 않는다

AI 가 "지금 수준이면 충분하다"고 프레이밍한 자리에서 **결론을 뒤집었습니다**. 프레이밍(이 과제의 평가축은 과정 깊이다)은 맞았지만, "그러니 여기서 멈춰도 된다"는 판단은 제 몫이었습니다. 축을 하나 더 없기로 한 그 결정에서 이 과제의 가장 어려운 부분이 나왔습니다.

### 규칙 5 — 범위 밖 추가는 몰래 있지 않는다

작업 중 "이것도 고치면 좋겠다"가 생겨도 합의 범위 밖이면 **명시하고** 진행했습니다. 예: 간헐 클로커의 거짓 복구를 막는 장치는 원래 범위 밖이라 커밋 메시지·문서·보고에 전부 "범위 밖 추가"라고 적었습니다.

---

## 4. 다시 한다면 무엇을 다르게 할까 (S2d)

### 4-1. 검사자에게 "버그 찾아줘"라고 묻지 않겠다

통제 실험으로 확인한 결과, "**이 주장을 반증해라**"가 "**결함을 찾아라**"보다 **확실히 강했습니다**. 후자는 탐색 공간이 무한이라 검사자가 어디를 팔지 스스로 정하고, 실제로 18 건을 내면서 정답은 한 건도 건드리지 못했습니다.

다시 한다면 **처음부터 주장을 명시적으로 적고** 그걸 반증시키겠습니다. 부수 효과가 하나 더 있습니다 — 반증할 명제를 쓰려면 내가 무엇을 보장한다고 믿는지 먼저 정리해야 합니다. **적지 않은 주장은 반증도 안 됩니다**.

#### 4-2. 라운드를 잘게 끊지 않겠다

구조화 리뷰를 한 번 돌렸더니 제가 **두 라운드에 걸쳐 나눠 받은 결함 4 건을 한 번에** 짚었습니다. 게이트를 여러 번 도는 게 곧 깊이가 아니었습니다. 라운드 수보다 **한 번에 얼마나 좁혀 묻느냐**가 결정적입니다.

#### 4-3. 측정 설계를 코드처럼 다루겠다

이번에 가장 많이 틀린 곳이 코드가 아니라 **재는 방법**이었습니다.

- 흔들어야 할 변수(시작 위상)를 고정한 채 재서 "오탐 0"이라는 틀린 결론을 낸
- 계측 도구가 만든 실패(포트 고갈)를 판정으로 집계해 수치가 조용히 좋아짐
- 검사자 실험 첫 회차에 **정답이 적힌 문서가 레포에 있어서** 검출 실험이 독해 시험이 됨

다시 한다면 측정 스크립트를 짤 때 "**이 실험에서 고정된 변수가 무엇인가**"를 먼저 적겠습니다. 그리고 확실적인 것을 "0"으로 보고하지 않겠습니다 — **상한과 표본 수**를 같이 적어야 합니다.

#### 4-4. 요약은 원문 대신 쓰지 않겠다

기록을 실시간으로 남긴 건 잘한 선택이었지만, 전부 **요약체**로 남겼습니다. 나중에 원문을 찾을 때 제 요약 표현으로 검색하다가 "원본이 소실됐다"는 결론까지 낼 뻔했습니다. 실제로는 표현이 달랐을 뿐 원문은 그대로 있었습니다.

과제 문서가 "날것 그대로 환영"이라고 쓴 이유가 이거라고 생각합니다. 다시 한다면 요약과 원문을 **처음부터 같이** 남기겠습니다.

#### 4-5. 완성도보다 "못 하는 것의 경계"에 시간을 쓰겠다

이번에 가장 만족스러운 산출물은 기능이 아니라 **README의 '알려진 한계'**입니다. 탐지 하한이 어디이고, 왜 거기이고, 그 아래에서는 어떻게 오동작하는지를 실측 표로 적었습니다.

그리고 그중 두 항목은 **20 라운드 게이트로도 못 잡고 검사자 실험에서야 나왔습니다** — 게이트는 "이 수리가 맞나"를 물었고 실험은 "이 도구가 뭐라고 주장하나"를 물었기 때문입니다. 질문이 다르면 나오는 것도 다릅니다.

---

## 5. 남은 것

- 클로킹 탐지 하한을 낮추려면 검색봇 시점 표본을 늘려야 하는데, 그건 감시 대상 사이트에 주는 부담과 직결됩니다. 지금은 그 균형점을 문서에 적어두고 멈춘 상태입니다.
- 시그니처 기반이라 등록되지 않은 문구는 못 잡습니다. 렌더된 가시 텍스트 비교로 바꾸면 우회에도 강해지지만 비용이 큼 — 로드맵으로만 남겼습니다.